Short Andrews-Curtis trivialisations for a hard benchmark of balanced presentations

Pragyaan Gaur

3 November 2026

Abstract

This note reports a computational attack on the Andrews-Curtis Conjecture Challenge of the SAIR Foundation and Caltech. The scored pool has 10115 balanced two-generator presentations of the trivial group, and a valid solution is a sequence of the official moves that carries the presentation to (x, y). We built a search engine for the official move set, checked it against the official verifier, and ran exact search together with two guided searches over the pool. 1354 of the 10115 presentations were solved on the ordinary track, of which 32 have a path that exhaustive search proves shortest. The same 1354 presentations were also submitted on the stable track by appending the two published destabilisation moves. We also prove in Lean two facts about the official statement that it uses without proof, namely that its reachability relation is an equivalence relation and that multiplication by an inverse relator is a derived move. The honest summary is that most of the pool resisted our methods. We describe the measurements that show why, so that the next attempt can start from evidence rather than from a guess.

1 The problem

A balanced presentation with two generators is an ordered pair of words (r_0, r_1) in the free group on x and y. The official move set has fourteen moves. A relator may be inverted, it may be multiplied on the right by the other relator or by its inverse, and it may be conjugated by one of the four letters x, x^{-1}, y and y^{-1}, with free reduction after every move. The ordinary track asks for a sequence of these moves that carries the pair to (x, y), in that order. The stable track allows the rank to grow to eight generators and asks for a sequence that empties the presentation.

Both tracks are scored by rank of path length. On each presentation, the teams tied for the shortest verified path share 2^{1-k} points, and every other team receives nothing. A single presentation is therefore worth one point to a team that holds the unique shortest path, and worth nothing at all to a team that holds a longer one. This scoring rule shaped everything below, because coverage without shortness earns no points.

2 Method

2.1 An engine checked three ways

The move set is small and exact, so the engine is a C program that stores a state as two freely reduced integer words. Correctness was fixed before any search was run. We wrote an independent Python implementation of the fourteen moves from the written specification rather than from the official source, and a third implementation is the official one. On four thousand random states and random move sequences the three agree exactly, and the eight published shortest suffixes from the

signed permutations of (x, y) each replay to the official target and are each minimal by exhaustive search. Every path produced by the engine is replayed inside the engine before it is written out, so a path whose endpoint is not exactly (x, y) is discarded rather than reported.

2.2 Exact search around the target

Search is only possible if something is known about the region near the target. We enumerated the ball of radius eight around the target by breadth first search, which gave 24587144 states, and stored each state's exact distance to the target. The ball grows by a factor of about eight and a half per level, so depth eight is close to the memory limit of the machine, and depth ten is out of reach.

The ball turns exact questions into decidable ones. Any path of length L has a vertex at distance $k = \max(0, L - 8)$ from the start, so a breadth first search from a presentation to radius r meets the ball for every path of length at most $8 + r$, and returns the true minimum. With $r = 5$ this certifies every path of length at most thirteen, and it also proves that no shorter path exists. This is the only part of the work that produces a proof of optimality.

2.3 Guided search for the rest

Most presentations are far outside the ball, so guided search is needed. We used a beam search over the fourteen moves with a score that is a weighted sum of the total relator length, the length of the longer relator, and the abelianised relator lengths. The beam stops when it reaches a state inside the ball, and the stored distance in the ball then supplies the tail of the path.

A second beam works in a coarser move set. A bare multiplication rarely cancels much, because the end of one random relator rarely matches the start of another. Rotating the other relator first can line up a long common subword and turn a useless multiplication into a large cancellation. The rotation is not free, since one conjugation by a letter rotates a word by one place, and the coarser move set therefore charges the rotation distance plus one for each macro. The idea of aligning by rotation before concatenating is taken from the Caltech ACSolverX code, which we read at commit 6a12515fe1d95178a483b76d5553266e61122417.

2.4 Shortening

Every presentation that more than one method can solve is solved several times, and only the shortest verified path is kept. The exact search produces the shortest path whenever the distance is at most thirteen, and it overrides every heuristic result in that range. Beyond that range the best of the runs is kept. Loop removal was implemented and run on every solved presentation. It removed nothing, because the beam keeps a visited set and therefore never returns to a state it has already seen, so the found paths are already loop free. We record that as a null result rather than as a technique that helped. The other shortening step in the plan, replacing the subpath between two fixed states by an optimal one, was not written, so our long paths stayed long.

2.5 Verification

No path enters a submission file without passing the official verifier. The consolidation step builds each part file, runs the official verifier on it, and keeps only the results that come back with `ok: true`. Each part file is then verified again on its own bytes, and the exit code and the number of accepted results are recorded in `submission/VERIFY.md`. Independent Python and C

implementations were cross checked against each other and against the official move table before any of this.

3 Results

3.1 Ordinary track

Of the 10115 scored presentations, 1354 were solved. The total number of moves across those solutions is 663718. Path lengths run from 8 to 1498, with median 416. Of the solved presentations, 32 have a path that the exact search proves shortest, so for those the question of optimality is settled and no competitor can do better than tie.

The submission is split into 3 part files for the ordinary track and 3 for the stable track, each within the five hundred solution limit.

3.2 Stable track

The stable track has its own leaderboard and its own target, and a solution to the ordinary track extends to a solution of the stable track by appending the two published destabilisation moves. We used exactly that extension, so the stable submission covers the same 1354 presentations at two moves more each. This is valid and it is not short. The stable move set allows the rank to grow, which buys real freedom, and we did not search that larger space. A better stable entry would search there directly.

4 What failed, and why

It is more useful to record the measurements that explain the failure than to describe the search settings.

The pool is not made of easy instances. Of the 10115 presentations, exactly 2 lie inside the ball of radius eight, so almost nothing is close to the target in the official metric. A breadth first search from each presentation to radius five, which certifies everything within thirteen moves, solved only a handful.

The pool is not made of random walks either. As a control we generated random walks from the trivial presentation of length up to thirty and ran the same search on their endpoints. The beam search recovered every one of them, and it usually found a path shorter than the walk that generated the instance. A random walk of length thirty is therefore easy for this method. The scored presentations are not of that kind.

The Miller-Schupp family is a minority of the pool by count, seven hundred and thirty three ids out of ten thousand one hundred and fifteen, but it accounts for almost every case that the literature regards as open. Reconstructing the family from the published encoding reproduces all four hundred and twenty four certified training presentations exactly, and the same reconstruction matches seven hundred and thirty three scored ids under the pool's own symmetry relation, which are exactly the five hundred and fifty rows that were open at the freeze, the one hundred and eighty three uncertified rows, and no certified row. That reproduces the organisers' statement about the composition of the pool and it checks the reconstruction end to end. Those are instances for which no trivialisation was known to the organisers or to the earlier literature. It is not surprising that they resist a three hour search.

The pool is also not excluded by the obvious invariant. The abelianised relation matrix of every one of the 10115 presentations has determinant ± 1, which is necessary for the presented group to

be trivial, so no abelian test can rule any of them out. That is consistent with the organisers' description of the pool and it means the difficulty is genuinely non-abelian.

Two further observations are worth stating. The length heuristic plateaus. On presentations it fails to solve, the beam reliably reduces the total relator length from around twenty seven to between fourteen and nineteen and then stops improving, and a state at that length is not close to the ball in any useful sense. Making the beam much wider, up to twenty thousand states, did not break the plateau on the presentation we used as a probe. The failure is structural rather than a matter of budget.

5 Lean results

The official statement file proves that the ordinary conjecture implies the stable one. It also uses two facts that appear only in comments. The first is that its reachability relation is an equivalence relation, on the grounds that every move can be undone. The second is that multiplication by an inverse relator is a derived move. Neither is proved in the repository, and the second matters for the competition, because the verifier demands the ordered pair (x, y) and the README justifies that by asserting that relator order is harmless.

We proved both. The Lean file shows that a single move is undone by a finite sequence, that inversion and conjugation need one move, and that right multiplication needs three because the move set never multiplies by an inverse relator directly. From that it derives that reachability is reflexive, symmetric and transitive, and it exhibits multiplication by an inverse relator as a three step derived move. A second Lean file shows that each of the fourteen rows of the verifier's frozen table is a single move or a three step composite of the published relation, which is the soundness direction of the correspondence between the verifier and the published conjecture. A third file, supplied by a referee after the first draft, proves that the two relators can be exchanged in nine moves at rank two, which is what justifies the ordered target the verifier demands. Two of the theorems in the second file depend on `Classical.choice` in addition to `propext` and `Quot.sound`, and the audit is stated per file for that reason. All of these are standard Lean axioms and none is an added one.

All three files compile with Lean 4.33.1. The audit reports `propext`, `Quot.sound` and, for three theorems, `Classical.choice`, with no `sorry`, no added axiom and no use of `native_decide`. The converse direction of the correspondence, that every published move is a finite sequence of verifier moves for an arbitrary conjugating word, is argued in prose in our notes but not formalised. No Lean theorem mentions the verifier's own word representation or its free reduction, so the bridge between the Lean statement and the verifier rests on a hand translation.

6 Limitations

This work does not touch the Andrews-Curtis conjecture. Finding paths for some presentations of the trivial group neither proves nor disproves the conjecture, and nothing here should be read as progress on that question. The stable track submission is an extension of the ordinary one, so it is not evidence about the stable conjecture either.

The coverage is partial and the authors say so plainly. The paths we hold for most solved presentations are long, and under a scoring rule that rewards only the shortest path they will earn nothing against a competitor who searches harder. The proved-optimal paths are the only results we can defend as final.

The search was run for a few hours on four cores. A serious attempt would need a trained policy over the coarse move set, and would search the stable space directly.

7 Use of AI tools

The search code, the verification harness, the Lean proofs and the prose of this note were produced by Open Math OS agents under the direction of the author. Pragyaan Gaur checked the work and is the sole author. No solution path appears in this note, and none will appear publicly before 1 December 2026.

References

[1] The Andrews-Curtis Conjecture Challenge. Official repository, `https://github.com/SAIRcompetition/Andrews-Curtis`, commit a0fd6e6f52d82c93ccc06ab91d81e8fa3678256e. Statement file `competition/tools/lean/AC.lean`, verifier specification `competition/tools/verifier/README.md`.

[2] Math-AI-Caltech, ACSolverX, `https://github.com/Math-AI-Caltech/ACSolverX`, commit 6a12515fe1d95178a483b76d5553266e61122417. Source of the rotation aligned concatenation move and of the AC19 and AC1M datasets.

[3] S. Akbulut and R. Kirby, *A potential smooth counterexample in dimension 4 to the Poincaré conjecture, the Schoenflies conjecture, and the Andrews-Curtis conjecture*, Topology 24 (1985).

[4] C. F. Miller III and P. E. Schupp, *Some presentations of the trivial group*, in Algorithms and Classification in Combinatorial Group Theory, MSRI Publications 23 (1992).

[5] A. Miasnikov and A. Myasnikov, work on computational and genetic search for Andrews-Curtis trivialisations, as surveyed in the dossiers under `notes/`.

[6] Shehper and coauthors, *What makes math problems hard for reinforcement learning: a case study*, and the accompanying solver repository, as cited with full identifiers in `notes/lit-computational-ml-ac.md`.